

Owner's Manual: Autonomous Crop-Health Monitoring System

1. Introduction

1.1 System Overview

The Autonomous Crop-Health Monitoring System is a specialized agricultural tool designed to provide high-resolution, real-time insights into greenhouse environments. By utilizing a modified Crazyflie 2.1 drone equipped with an AI Deck and Lighthouse positioning technology, the system autonomously navigates indoor rows to identify early signs of **crop stress**. By catching physiological changes early, the system leverages ML to inform whether growers should intervene before yield is compromised.

1.2 The Goal: Maximizing Yield

In a modern greenhouse, early detection is the difference between a minor adjustment and a lost crop. This system was developed with one primary objective: **Maximizing Yield**.

Traditional manual scouting is time-consuming and prone to human error. By automating the scouting process, this system offers:

- **Consistency:** Every plant is checked on a precise schedule without fatigue.
- **Objectivity:** AI-driven analysis provides data-backed stress assessments, removing the guesswork from visual inspections.
- **Speed:** The drone surveys large greenhouse areas in a fraction of the time required for a person to walk the rows.
- **Precision:** High-fidelity sensors detect "hidden" stress markers that the human eye might miss until damage is irreversible.

1.3 Key Features

- **Fully Autonomous Navigation:** Uses Lighthouse base stations for centimeter-level positioning accuracy.
- **Real-Time Data Streaming:** Live video and telemetry feeds are sent directly to your host machine over a dedicated 2.4GHz network.
- **Advanced Health Analytics:** On-board algorithms process raw sensor data to provide smoothed, actionable stress-detection maps.
- **User-Friendly Dashboard:** A centralized front-end to monitor your greenhouse status at a glance.

1.4 What's in the Box

Your system arrives pre-configured. Please ensure the following components are included:

Component	Quantity	Description
-----------	----------	-------------

Crazyflie 2.1 Drone	1	Fully assembled with AI Deck and Lighthouse Deck.
LiPo Batteries	2	High-capacity flight batteries.
Battery Charger	1	Dedicated USB-powered charger for flight batteries.
Lighthouse Base Stations	2	V2 units for indoor positioning.
Crazyradio PA	1	High-gain 2.4GHz USB radio for drone communication.
MicroUSB Cable	1	For connecting the radio or drone to the host.
Spare Propellers	1 Set	Complete set of clockwise and counter-clockwise props.
Extra Motor Arms	2	Replacement plastic frame arms.
Extra Motor	1	Spare coreless DC motor.

2. Safety and Compliance

2.1 General Safety Rules

- **Indoor Use Only:** The Crazyflie is ultra-lightweight and is not designed to combat wind. Only operate within the greenhouse or enclosed structures.
- **Avoid High-Traffic Areas:** Do not fly the drone directly over people. Ensure the flight path is clear of staff during scheduled missions.
- **Environmental Check:** Ensure there are no loose hanging wires, strings, or plastic sheeting in the flight zone.

2.2 Propeller Safety

- **Rotating Parts:** The motors spin at high RPMs. Never touch the drone while propellers are spinning.
- **Propeller Inspection:** Before every flight, check for nicks or cracks. Damaged blades cause sensor-disrupting vibrations.

2.3 Automated Safety Features (Flight Script)

- **Geofencing:** Restricted to pre-defined 3D boundaries (X, Y, Z).
- **Lighthouse Monitoring:** Initiates "Return to Home" (RTH) upon signal loss or unstable positioning data.
- **Battery Protection:** Auto-returns to the origin if battery levels drop below **10 percent**.

2.4 Emergency Procedures (Abort/Kill Switch)

- **Immediate Abort:** Press **Ctrl + C** on the keyboard to land immediately.
- **Power Disconnect:** If safe, disconnect the LiPo battery immediately after a crash.
- **Recovery:** Use the “**Recover**” button in the bottom left of **Cfclient** if the drone becomes unresponsive after an impact.

3. Software Environment Installation

Setting up the software correctly is the first step toward a successful flight. To maintain a stable system, we use a **Virtual Environment** to isolate the drone software from your other computer files.

3.1 Host Requirements

- **OS:** Windows 10/11 or macOS.
- **Python:** Version 3.11 is required.
- **Hardware:** One available USB port for the Crazyradio PA.

3.2 Step 1: Install Python 3.11 (Global Setup)

You must install Python on your system before you can create a virtual environment.

For Windows:

1. Download the Python 3.11 installer from python.org.
2. Run the installer. **IMPORTANT:** You must check the box that says “**Add Python to PATH**” before clicking “Install Now.”
3. Once finished, open PowerShell and type `python --version` to confirm.

For Mac:

1. Install Homebrew (if you don't have it) by pasting this into Terminal:
`/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"`
2. Install Python 3.11 and libusb:
`brew install python@3.11 libusb`
3. Confirm by typing `python3.11 --version` in Terminal.

3.3 Step 2: Configure Virtual Environment & Workspace

1. Create a folder named `greenhouse_drone` in your Documents.
2. Open your terminal (PowerShell for Windows, Terminal for Mac) and navigate to that folder:
`cd Documents/greenhouse_drone`

3. Create the Environment:

python -m venv venv (Windows) or python3.11 -m venv venv (Mac)

4. Activate the Environment:

- **Windows:** .\env\Scripts\activate
- **Mac:** source venv/bin/activate
- *The prefix (venv) should now appear in your terminal prompt.*

3.4 Step 3: Windows Driver Setup (Zadig)

Windows users must manually configure the USB driver for the Crazyradio PA.

1. Download **Zadig** from zadig.akeo.ie.
2. Plug the Crazyradio PA into your computer.
3. Open Zadig and go to Options -> List All Devices.
4. Select Crazyradio PA from the dropdown menu.
5. In the "Driver" section, ensure **libusb-win32** is selected in the right-hand box.
6. Click "**Replace Driver**".

3.5 Step 4: Install Drone Software (Inside Venv)

With your virtual environment active, install the Crazyflie client and dependencies:

- pip install cfclient

3.6 Step 5: Initial Live Connection Test

1. **Power On:** Turn on the Crazyflie with a battery installed.
2. **Launch Client:** Inside your terminal (venv active), type cfclient.
3. **Check Radio:** Confirm the Crazyradio PA is detected in the dropdown in the upper left.
4. **Verify Live Data:**
 - Click **Connect**.
 - Pick up and tilt the drone; the **Artificial Horizon** in the client should respond.
 - Move the drone; verify the **Positioning Data** (X, Y, Z) updates in real-time.

4. Hardware Setup and Calibration

4.1 Battery Installation

The AI Deck acts as the battery hold-down. Slide the LiPo battery into the cavity **between the AI Deck and the mainboard**. Connect the white plug to the power port, ensuring wires are tucked away from the propellers.

4.2 Lighthouse Base Station Setup

1. **Placement:** Mount two base stations in opposite corners of the greenhouse at **2 meters** height.
2. **Orientation:** Point both stations directly at the **Origin** (the center of the flight area).
3. **Channels:** * Connect each station to your computer via USB.
 - Using **Cfclient**, navigate to the Lighthouse Positioning tab and click "Set BS Channel". Set one base station to Channel 1, then plug in the second and set it to Channel 2.

4.3 Calibration Workflow

- **Initialize:** Open **Cfclient** -> **Lighthouse Positioning Tab**.
- **Estimate:** Place the drone at the **Origin**. Click **Manage Geometry** -> **Estimate Geometry**. Follow prompts.
- **Verify:** Return the drone to the Origin. X, Y, and Z in the client should read near 0,0,0.

4.4 Stability and Performance Testing

- **Hover Test (hover_test.py):** * Activate your venv and run: `python hover_test.py`.
 - Run at **2ft** and **4ft**.
 - **Verification:** The drone must hover steadily. If it wobbles, repeat Section 4.3 calibration.
- **Flight Test (flight_test.py):** * Executes the scan pattern while printing raw sensor data to the terminal.
 - Use this to verify that camera and distance sensors are responding accurately.
 - Press **Ctrl + C** to end the test and land.

5. Deployment and Operating Procedures

5.1 Daily Operating Procedure (DOP)

To ensure long-term system health and reliable data, follow this routine every day the system is in use:

Morning Startup:

1. **Inspect Greenhouse:** Walk the rows to ensure no new obstructions (tools, hanging plastic, or overgrown foliage) have entered the flight path.
2. **Power Base Stations:** Plug in both Lighthouse Base Stations. Confirm the green status light is solid on both units.
3. **Battery Prep:** Remove a fully charged battery from the charger. Check for any swelling or damage.
4. **Boot Host System:** Power on the host computer and open the terminal in your `greenhouse_drone` folder. Activate the virtual environment.

End-of-Day Shutdown:

- **Power Down Drone:** Disconnect the battery from the Crazyflie.
- **Charge Batteries:** Place used batteries on the charger. Never leave batteries at 10% charge overnight.
- **Power Down Base Stations:** Unplug the Lighthouse stations to prolong the life of the internal motors.
- **Data Sync:** Ensure all data collected during the day has been successfully uploaded to the dashboard (see Section 8).

5.2 Operator Pre-Flight Checklist

Before every mission, the operator **must** verify the following points. Failure to do so may result in a crash or inaccurate data.

1. ☐ **Propellers:** Firmly attached and free of nicks or cracks.
2. ☐ **Battery:** Slotted securely between the AI Deck and Mainboard; connector is tight.
3. ☐ **Clearance:** Flight path is clear of personnel and temporary obstacles.
4. ☐ **Software Connection:** cfclient shows a stable radio connection with 0% packet loss.
5. ☐ **Positioning Lock:** Lighthouse tab in cfclient shows a "Green" status and accurate 0,0,0 coordinates at the Origin.
6. ☐ **Sensors:** AI Deck camera feed is visible and clear of dust/smudges.

5.3 Executing a Mission

Once the checklist is complete, you are ready to launch the autonomous monitoring mission.

1. **Start the Dashboard:** Run the front-end server command in a separate terminal window (see Section 8).
2. **Launch Mission:** In your main terminal (venv active), run the mission script:
`python mission.py`
3. **Monitor Progress:** Keep the host computer within line-of-sight of the drone. Watch the dashboard for live crop-stress updates.
4. **Supervise:** Be prepared to press **Ctrl + C** at any moment if an emergency occurs. The drone will automatically land and terminate the mission.
5. **Post-Mission:** Once the drone returns to home and lands, wait for the motors to stop completely before approaching to swap batteries or power down.

6. System Reconfiguration & Network Management

When moving the system to a new greenhouse zone or changing your local network, follow these advanced reconfiguration steps.

6.1 Relocating to New Spaces

6. **Physical Move:** Relocate the Lighthouse Base Stations to the new zone. Follow the height and orientation guidelines in Section 4.2.
7. **Redefine Boundaries:** Open `flight_config.py` in your code editor. Modify the `MAX_X`, `MAX_Y`, and `MAX_Z` parameters to match the dimensions of the new flight area.
8. **Re-Calibration:** You **must** perform a full Geometry Estimation (Section 4.3) every time a base station is moved.

6.2 Changing the Wireless Network (AI Deck)

To update the SSID or Password, you must re-flash the drone's firmware. The most reliable method across all systems is using the **Bitcraze Docker Toolbelt**.

A. Docker Installation

1. **Windows:** Download and install **Docker Desktop** from docker.com. Ensure WSL 2 is enabled during installation.
2. **Mac (Intel):** Download and install the **Intel Chip** version of Docker Desktop.
3. **Mac (M-Series):** Download and install the **Apple Chip** version of Docker Desktop.

B. Configure Credentials (nconfig)

Navigate to your crazyflie-firmware folder in your terminal and run:

Windows (PowerShell):

```
docker run --rm -it -v ${PWD}:/module bitcraze/toolbelt make nconfig
```

Mac (Intel or Apple Silicon):

```
docker run --rm -it -v ${PWD}:/module bitcraze/toolbelt make nconfig
```

- Navigate to Expansion deck configuration -> Support the AI-deck.
- Set Wifi setup at startup to **Connect to a Wifi network**.
- Enter your SSID and Password.
- Press **F6** to Save and **F9** to Exit.

C. Build and Flash

1. **Build:** Run the same docker command but replace `make nconfig` with `make`.
2. **Flash Mainboard (STM32):** Put drone in bootloader mode (hold power 3s).
`python3 -m cfloader flash build/cf2.bin stm32-fw`
3. **Flash AI Deck (GAP8):** Power drone normally (not in bootloader mode).
`python3 -m cfloader flash [path_to_gap8_img] deck-bcAI:gap8-fw`

7. Data Collection and Processing

Understanding the data the system collects and how it is processed is key to interpreting the health of your crops.

7.1 Sensor Suite & ML Model Inputs

The drone collects four primary data streams which are interpreted independently by the Machine Learning (ML) model to predict crop stress:

- **Images (Camera):** The model analyzes the camera feed to identify subtle changes in **crop color**, which is a primary indicator of physiological stress.
- **Temperature (Raw & Corrected):** The system tracks ambient temperature. High-heat environments can induce thermal stress in plants. The "Corrected" value accounts for motor heat to ensure high precision.
- **Atmospheric Pressure:** Changes in pressure can signal environmental shifts that affect plant transpiration rates.
- **Air Density:** Calculated based on pressure and temperature, air density is used to correlate flight efficiency with environmental stress markers.

Crucially, each of these variables influences the ML model's predictions separately. The model does not use temperature to interpret color; instead, it looks at the combination of visual and environmental data to form a holistic stress assessment.

7.2 Unit Selection & Launch Prompt

When you launch the flight script, you will be prompted to provide an ambient reference:

"Please enter ambient temperature at starting position (e.g., 25C or 77F):"

- **Accuracy is Critical:** Ensure the ambient temperature you enter is as accurate as possible. It should represent the average temperature of the room, especially if different zones in your greenhouse vary in heat.
- **Unit Selection:** Entering a capital **C** or **F** after the number sets the units for the entire session.
- **Reference Data:** This value is used as a reference to calibrate the raw temperature sensor and apply the smoothing algorithm.
- **Data Saving:** The software will automatically save all subsequent data in the units you selected (e.g., entering 70F will save logs with Fahrenheit headers).

7.3 Data Storage (drone_mapping_data.csv)

Every data point captured during flight is saved locally to a file named `drone_mapping_data.csv` in the directory where the program is run.

- **File Management:** The Front-End Dashboard reads data directly from this CSV file when not in live mode. **Do not move or rename this file** if you wish to use the dashboard for historical data analysis.

- **Recorded Parameters:** Based on your selected units (Fahrenheit example shown), the following parameters are logged:
 - **index:** Sequential reading number.
 - **datetime_et:** Timestamp of the data capture.
 - **state/mode/event:** Current flight status and tracking events.
 - **roc_F_per_s:** Rate of Change for temperature readings.
 - **raw_temp_F / expected_raw_F / deviation_F:** Temperature sensor metrics before smoothing.
 - **corr_temp_F:** The final, motor-corrected temperature used for analysis.
 - **thermal_offset_F / ambient_est_F:** Calibration offsets for the internal thermistor.
 - **rel_alt_m / rel_alt_ft:** Current altitude in meters and feet.
 - **air_density_kg_m3:** Calculated air density at current altitude.
 - **station_pressure_hPa / sea_level_pressure_QFF_hPa:** Barometric pressure readings.
 - **battery_V:** Current drone battery voltage.
 - **thrust:** Motor output level.

8. The Front-End Dashboard

The **CropWatcher** Dashboard is your centralized command and analysis center. It is designed to be user-friendly, translating complex drone telemetry and ML predictions into actionable greenhouse insights.

8.1 Modes of Operation

- **Live Mode:** Used during a flight. Requires `telemetry_api_server.py` to be running. This mode provides real-time gauges and live tracking.
- **Historical Mode:** Used for review. Does **not** require the API server. Simply open the `mainFrontEnd_live.html` file and use the **"Upload CSV"** button to load past missions.

8.2 Detailed Tab Breakdown

The interface is organized into ten specialized tabs. To navigate, use the sidebar on the left side of the dashboard.

1. Dashboard

This is your high-level overview page.

- **Key Statistics:** Displays the "Last Scouting Time," total plants scanned, and the current number of high-stress alerts.
- **Global Health Tile:** A large color-coded indicator (Green/Yellow/Red) that signals the

overall condition of the greenhouse.

- **System Connectivity:** Shows whether the API server is connected and the status of the drone's radio link.

2. Zone Health Map

The primary spatial analysis tool.

- **Spatial Heatmap:** Generates a 2D map where colors represent crop health levels. **Deep Red** areas indicate severe stress, while **Green** indicates healthy growth.
- **Interactive Grid:** Hover over any zone to see the specific coordinates and the most recent ML stress score for that area.

3. Time Series

Visualizes environmental and health trends over the duration of a mission.

- **Multi-Metric Graphs:** Plots **Corrected Temperature**, **Pressure**, and **Stress Scores** against the mission timeline.
- **Trend Analysis:** Use this tab to identify whether stress levels are increasing or if environmental fluctuations correlate with specific flight segments.

4. Flight Path

Tracks the drone's physical movement within the greenhouse boundaries.

- **Coordinate Plane:** A top-down grid representing your floor space. A moving point shows the drone's live (X, Y) position.
- **Geofence Boundary:** Displays the pre-set safety box. If the drone nears an edge, the boundary on the map will glow red.
- **Coverage Tracking:** Useful for ensuring that the drone has successfully traversed all intended rows.

5. Stress Forecast

Utilizes the Machine Learning model to project future stress points.

- **Predictive Plotting:** Based on current data trends (rising temperature combined with color shifts), this tab flags plants that are likely to enter high-stress states within the next scout cycle.
- **Confidence Scores:** Displays the ML model's certainty for each stress prediction.

6. Raw Data Table

A granular, tabular view of every data packet collected.

- **Data Inspection:** View the exact numerical values for every column in the CSV (Altitude,

Thrust, Voltage, etc.) for any specific timestamp.

- **Export Filter:** Allows you to filter the table to show only "High Stress" rows before exporting.

7. Run Comparison

The primary tool for tracking greenhouse progress over time.

- **Comparative Mapping:** Allows you to upload two different CSV files (e.g., Monday vs. Friday).
- **Difference View:** Highlights specific coordinates where plant health has improved or declined between the two scouting missions.

8. Live Telemetry

Requires the **API Server** and provides high-fidelity monitoring during active missions.

- **Real-Time Gauges:** Digital readouts for **Battery Voltage**, **Air Density**, and **Corrected Temperature**.
- **Flight Logic:** Shows the current internal state (e.g., MAPPING, STABLE, RTH) to keep the operator informed of the drone's automated decisions.

9. Camera Feed

Provides the direct visual perspective from the AI Deck.

- **Grayscale Stream:** Displays the high-speed canopy feed.
- **Video Diagnostics:** Shows the current FPS and connection latency. If the feed is black, refer to the WiFi setup in Section 6.

10. Alerts

A centralized log of all significant system and health events.

- **Stress Alerts:** Timestamped list of every (X, Y) coordinate where high stress was detected.
- **System Warnings:** Flags low battery levels, geofence violations, or Lighthouse signal drops.
- **Staff Export:** Use the "Export Stress List" button to give greenhouse staff a printed list of problem coordinates for manual inspection.

9. Troubleshooting

This section lists common issues encountered by greenhouse operators and the technical steps

to resolve them.

9.1 Connection & Driver Issues

- **"Radio not found" in Cfclient:**
 - **Fix:** Ensure the Crazyradio PA is plugged in firmly. Make sure the drone is powered on. If on Windows, run **Zadig** and confirm the driver is set to **libusb-win32**. If on Mac, ensure "Python" has permission to access USB devices in System Settings.
- **"API Disconnected" on Dashboard:**
 - **Fix:** Check that the `telemetry_api_server.py` terminal window is still active. If the terminal shows an "Address already in use" error, close all terminal windows and restart the script.
- **Connection Drops during Flight:**
 - **Fix:** Ensure you are using a 2.4GHz WiFi network that is not heavily congested. Move the host computer closer to the flight zone to improve the Crazyradio signal.

9.2 Flight Stability & Sensor Interference

- **Drone Drifts or Crashes into Boundaries:**
 - **Fix:** This is almost always a calibration issue. Re-run the **Estimate Geometry** workflow (Section 4.3). Ensure both Lighthouse stations are on different channels and are pointed toward the Origin.
- **Lighthouse Tab stays "Red":**
 - **Fix:** Clean the infrared sensors on the Lighthouse Deck with a microfiber cloth. Check for physical obstructions (foliage or support beams) blocking the line of sight between the stations and the drone.
- **Tumble on Takeoff:**
 - **Fix:** Verify propeller orientation. Clockwise (CW) propellers must be on the Front-Left and Back-Right motors. Counter-Clockwise (CCW) propellers must be on the Front-Right and Back-Left.

9.3 Common LED Error Code Reference

Pattern	Meaning	Action
Solid Red (M1)	System Error	Restart the drone; check expansion decks.
Flashing Red (M1)	Low Battery	Charge or swap the LiPo battery.
Flashing Blue (M4)	Bootloader Mode	Flash firmware or tap power once to exit.

Pulsing Green	Ready for Flight	Normal operation; proceed with mission.
----------------------	------------------	---

10. Maintenance and Support

To ensure the longevity of your system, follow this regular maintenance schedule.

10.1 Physical Maintenance

- **Camera Lens Cleaning:** Use a small drop of lens cleaner and a soft swab to clean the AI Deck lens every 5 missions. Dust on the lens can cause the ML model to misinterpret crop color.
- **Motor & Propeller Check:** After every 20 flight minutes, inspect the **plastic motor arms** for hairline fractures. Check props for "stress whitening" (areas where the plastic has turned white due to bending) and replace them immediately.
- **Battery Storage:** If the system will not be used for more than two weeks, charge the batteries to approximately $3.85V$ (Storage Charge) rather than leaving them empty or fully charged.

10.2 Software & Firmware Updates

- **Environment Updates:** Occasionally run `pip install --upgrade cfclient` inside your virtual environment to ensure you have the latest stability patches from Bitcraze.
- **ML Model Refinement:** If you notice consistent false-positives on the dashboard, contact support for a model update. New weights can be flashed via the Docker workflow detailed in Section 6.

10.3 Component Replacement Guide

- **Propeller Swap:** Pull the old prop straight up to remove. Ensure the new prop is the correct direction (CW/CCW) before pushing it down onto the motor shaft.
- **Motor Replacement:** The motors are plug-and-play. Disconnect the white motor plug from the mainboard and slide the motor out of the plastic arm. Insert the spare motor and secure the wiring along the arm channel.